Продвинутое машинное обучение

Лекция 5: Современные GBDT и feature engineering: Taylor и Newton boosting, XGBoost, LightGBM, CatBoost, target encoding

Автор курса: Лапшина Анастасия (University of Maryland, College Park, ВШЭ)

НИУ ВШЭ 2026-2027

Часть 1. Второй порядок: Taylor и Newton boosting

Где мы остановились и актуальные обозначения

Лекция 4: функциональный градиентный спуск

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

где поправка обучается на псевдо-остатках

$$r_{m,i} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

- Псевдо-остатки - это антиградиент риска, то есть первая производная loss
- Величина шага не следует из градиента: её подбирает line search
- Loss участвует ровно один раз — при вычислении первой производной

Обозначения этой лекции

f□(x)	базовая модель шага m (дерево)
g_i	первая производная loss по предсказанию
h_i	вторая производная loss по предсказанию
I□	множество объектов, попавших в лист j
w□	значение, которое дерево выдаёт в листе j
T	число листьев дерева
η	learning rate (shrinkage)

Базовую модель теперь обозначаем f, а не h: буква h занята под вторую производную. Так пишут в статьях XGBoost, LightGBM и CatBoost.

Разложение функции потерь в ряд Тейлора

Ансамбль F_{m-1} уже построен и зафиксирован. Ищем поправку f. Раскладываем loss по f в точке $F_{m-1}(x_i)$:

$$L(y_i,\, F_{m-1}(x_i) + f(x_i)) \approx L(y_i, F_{m-1}(x_i)) + g_i\, f(x_i) + \frac{1}{2}\, h_i\, f(x_i)^2$$

$$g_i = \left[\frac{\partial L(y_i, F)}{\partial F}\right] F = F_{m-1}(x_i) \qquad h_i = \left[\frac{\partial^2 L(y_i, F)}{\partial F^2}\right] F = F_{m-1}(x_i)$$

Первый член

$L(y_i, F_{m-1}(x_i))$ от f не зависит - это константа шага, её отбрасываем.

Оставим только g_i

Получится gradient boosting - то, что мы разбирали в лекции 4.

Оставим g_i и h_i

Получится Newton boosting - основа XGBoost, LightGBM и CatBoost.

То есть разложение отвечает на следующий вопрос: если я добавлю к текущему предсказанию поправку $f(x_i)$, как изменится потеря на объекте xi (на сколько она сдвинется и в какую сторону)?

Первый порядок: почему нужен line search

Оставим в разложении только линейный член. Минимизировать сумму $g_i \cdot f(x_i)$ напрямую нельзя: она не ограничена снизу — достаточно взять f сколь угодно большим со знаком $-g_i$.

$$f^*(x_i) \propto -g_i$$

направление известно, масштаб — нет

Как это чинят на практике

- Ограничиваем класс базовых моделей: дерево фиксированной глубины не может дать бесконечную поправку
- Обучаем дерево методом наименьших квадратов на таргет $-g_i$
- Масштаб подбираем отдельно - line search по γ

Что при этом теряется

- Кривизна loss игнорируется: объекты, где loss почти плоский, и объекты, где он резко растёт, получают одинаковый вес
- Один общий множитель γ на всё дерево - шаг не адаптируется к отдельному листу
- Line search - это ещё одна одномерная оптимизация на каждой итерации

Newton boosting: добавляем кривизну

Оставляем оба члена. Отбросив константу, получаем квадратичную по f задачу:

$$\tilde{\mathcal{L}}^{(m)}(f) = \sum_{i=1}^{n} \left[g_i \, f(x_i) + \frac{1}{2} h_i \, f(x_i)^2 \right]$$

Выделим полный квадрат — задача превращается во взвешенную регрессию:

$$\tilde{\mathcal{L}}^{(m)}(f) = \frac{1}{2} \sum_{i=1}^{n} h_i \left(f(x_i) - \left(-\frac{g_i}{h_i} \right) \right)^2 + \text{const}$$

Таргет

$$f^*(x_i) = -\frac{g_i}{h_i}$$

шаг Ньютона для объекта

Вес объекта

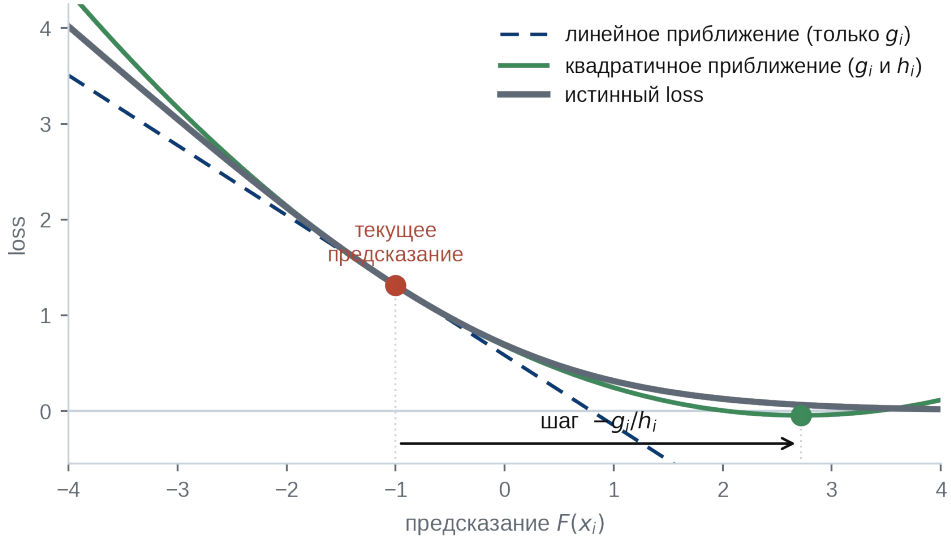
h_i

чем сильнее изгиб loss, тем важнее объект

Условие применимости

Нужна $h_i > 0$, то есть выпуклый loss. У MAE вторая производная нулевая почти всюду - чистый Newton там не работает.

Геометрия шага и почему гессиан - это число



Парабола касается кривой в текущей точке и повторяет её кривизну. Именно её минимум и даёт ньютоновский шаг.

Что видно на графике

- Квадратичную задачу можно решить точно, в одну формулу
- Линейное приближение уходит вниз без ограничения - минимизировать его нельзя
- Минимум параболы уходит ниже нуля: приближение верно локально, отсюда сжатие шага через η

Почему h_i - число, а не матрица

Риск - сумма по объектам, и лосс объекта i зависит только от его собственного предсказания z_i:

$$\frac{\partial^2 R}{\partial z_i \, \partial z_j} = 0 \qquad (i \neq j)$$

матрица n×n схлопывается в свою диагональ

$$\mathbf{H} = \operatorname{diag}(h_1, h_2, \ldots, h_n)$$

На графике: логистический loss при y = 1 и текущем логите F = −1, где h = p(1−p) ≈ 0.20. Малая кривизна - большой шаг.

Значение листа и два знакомых частных случая

Дерево выдаёт одну константу w□ на весь лист. Минимум взвешенного квадратичного критерия внутри листа – взвешенное среднее ньютоновских шагов:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i}$$

Квадратичный loss

$$L = \tfrac{1}{2}(y - F)^2 \;\Rightarrow\; g_i = F - y_i, \quad h_i = 1$$

Все веса равны единице, значение листа — обычное среднее остатков. Gradient и Newton boosting здесь совпадают: на регрессии с MSE разницы не видно.

Логистический loss

$$L = \log(1 + e^{-yF}) \;\Rightarrow\; h_i = p_i(1 - p_i)$$

Вес максимален при p ≈ 0.5 и падает у уверенных предсказаний. Модель концентрируется на границе разделения — здесь второй порядок и выигрывает.

Ср. с лекцией 4: там значение листа для MSE выводилось напрямую – это тот же результат при $h_i = 1$.

Gradient boosting и Newton boosting: отличия

	Gradient boosting	Newton boosting
Порядок разложения	первый	второй
Таргет базовой модели	$-g_i$	$-g_i / h_i$
Веса объектов	одинаковые	h_i (кривизна loss)
Величина шага	один γ на дерево, line search	своя для каждого листа, из формулы
Требование к loss	дифференцируемость	выпуклость, $h_i > 0$
Где встречается	sklearn GradientBoosting, классический GBM	XGBoost, LightGBM, CatBoost

На практике различие заметно там, где кривизна loss сильно меняется от объекта к объекту: классификация, ранжирование, пуассоновская регрессия.

Часть 2. XGBoost: регуляризация в критерии разбиения

Регуляризованная целевая функция

К эмпирическому риску добавляется штраф за сложность каждого дерева ансамбля:

$$\text{Obj} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{M} \Omega(f_k)$$

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|$$

γ **штраф за каждый лист**

Разбиение делается, только если выигрыш перекрывает γ. Это pre-pruning, встроенный в критерий.

λ **L2 на значения листьев**

Стягивает w□ к нулю. Сильнее всего действует на листья с малым числом объектов.

α **L1 на значения листьев**

Может занулять листья целиком. Используется реже, чем λ.

Оптимальное значение листа и structure score

Дерево постоянно внутри листа, поэтому сумму по объектам можно пересобрать в сумму по листьям:

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^{T} \left[G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2 \right] + \gamma T$$

$$G_j = \sum_{i \in I_j} g_i \qquad H_j = \sum_{i \in I_j} h_i$$

Значение листа

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Это ньютоновский шаг, к знаменателю которого добавлена λ: чем меньше объектов в листе, тем сильнее сжимается его значение.

Structure score

$$\tilde{\mathcal{L}}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T$$

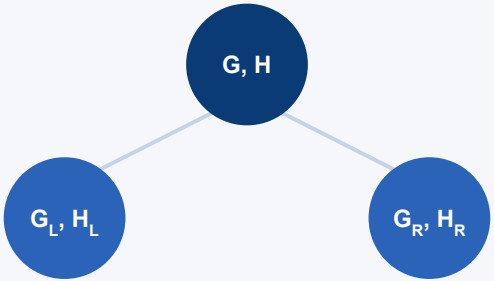
Подставили w□* обратно. Величина зависит только от структуры разбиения — это аналог impurity, выведенный из loss, а не постулированный.

Чем меньше значение structure score, тем лучше структура дерева.

Критерий разбиения: Gain

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

score(левый) + score(правый) − score(родителя) – штраф за появление нового листа



Суммы градиентов и гессианов аддитивны: сумма по родителю равна сумме по детям. Поэтому один проход по признаку пересчитывает Gain для всех порогов.

Разбираем компоненты:

- Gain < 0 – разбиение не делается: γ работает как порог полезности, а не как отдельная процедура обрезки
- λ в знаменателях штрафует листья с малой суммой гессианов, то есть слабо населённые
- min_child_weight ограничивает H в листе напрямую – это не число объектов, а сумма h_i

XGBoost всё же строит дерево до max_depth и обрезает назад: локально отрицательный Gain может окупиться более глубоким разбиением.

Как перебирать пороги: exact greedy и approximate

1 Exact greedy

- Сортируем объекты по значению признака и идём одним проходом, накапливая G и H слева
- Перебираются все существующие в данных пороги - разбиение точное
- Сортировка делается один раз и переиспользуется (column block)
- Не масштабируется: нужны все данные в памяти, плохо распределяется

2 Approximate + weighted quantile sketch

- Вместо всех порогов берём кандидатов-квантили признака
- Квантили взвешены гессианами - вклад объекта в задачу пропорционален h_i
- global: кандидаты считаются один раз на дерево; local: пересчитываются в каждом узле
- Даёт распределённость и работу с данными, не влезающими в память

$$r_k(z) = \frac{1}{\sum_i h_i} \sum_{i:\, x_{ik} < z} h_i$$

Ранговая функция, по которой ищутся кандидаты: доля суммарного веса h_i слева от порога. Здесь видно, что «важность» объекта при выборе сетки — это его кривизна, а не единица.

Часть 3. LightGBM: гистограммы, GOSS и EFB

Гистограммный поиск разбиений

Непрерывные признаки один раз разбиваются на бины (обычно 255). Дальше алгоритм работает с целочисленными номерами бинов, а не с исходными значениями.

Что меняется в стоимости

$$O(n \cdot d) \longrightarrow O(\#\text{bins} \cdot d)$$

- Сортировка не нужна вообще: строим гистограмму сумм G и H по бинам
- Перебор порогов идёт по числу бинов, а не по числу объектов
- Память падает: вместо float хранится однобайтовый номер бина
- Границы бинов строятся по обучающей выборке до начала бустинга

Приём вычитания гистограмм

$$\text{hist}_{\text{child}_2} = \text{hist}_{\text{parent}} - \text{hist}_{\text{child}_1}$$

- Считаем гистограмму только для того ребёнка, где меньше объектов
- Второй получаем вычитанием из родительской - бесплатно
- Экономия примерно вдвое на каждом уровне дерева

Порог ставится только на границе бина (небольшая потеря точности порога). На практике это мягкая регуляризация - модель не цепляется за отдельные значения.

Гистограммный режим есть и в XGBoost (tree_method=hist), и в CatBoost

GOSS: сэмплирование по величине градиента

Объекты с малым $|g_i|$ уже хорошо предсказаны и почти не влияют на Gain. Но выбросить их совсем нельзя – изменится распределение данных.

1 Сортируем

Упорядочиваем объекты по убыванию $|g_i|$.

2 Берём верх и случайный хвост

Оставляем верхние a·100 % и случайные b·100 % из остальных.

3 Компенсируем

Градиенты случайной части умножаем на множитель – иначе оценка Gain смещается.

$$\frac{1-a}{b}$$

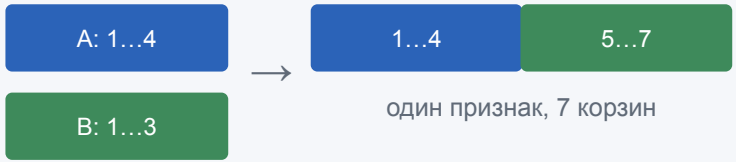
$$\tilde{V}_j(d) = \frac{1}{n}\left(\frac{\left(\sum_{A_l} g_i + \frac{1-a}{b}\sum_{B_l} g_i\right)^2}{n_l(d)} + \frac{\left(\sum_{A_r} g_i + \frac{1-a}{b}\sum_{B_r} g_i\right)^2}{n_r(d)} \right)!$$

Оценка прироста по подвыборке: вклад «лёгких» объектов масштабируется так, чтобы в среднем восстановить сумму по всем данным.

EFB: склейка взаимоисключающих признаков

В разреженных данных (one-hot, счётчики слов) многие признаки почти никогда не бывают ненулевыми одновременно. Такие признаки можно хранить и обрабатывать как один.

Как склеиваются диапазоны



К значениям второго признака добавляется сдвиг, равный числу корзин первого. Диапазоны не пересекаются, поэтому по номеру корзины всегда понятно, о каком исходном признаке идёт речь.

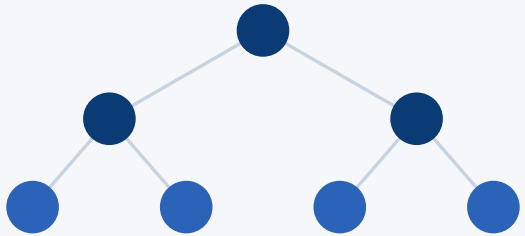
Как ищутся бандлы

- Строим граф: вершины — признаки, рёбра — частота одновременной ненулевой встречаемости
- Задача разбиения на бандлы эквивалентна раскраске графа и NP-трудна
- Используется жадная эвристика с допустимым уровнем конфликтов: небольшое пересечение разрешено
- Итог: сложность падает с числа признаков до числа бандлов

EFB даёт выигрыш ровно там, где данные разрежены; на плотных числовых таблицах эффекта почти нет.

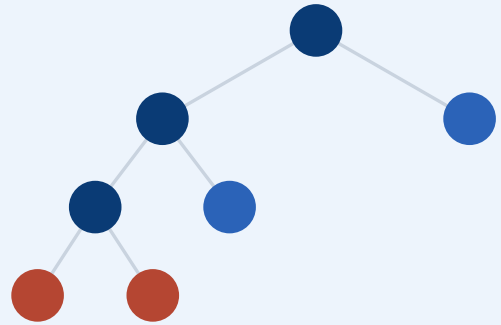
Порядок роста дерева: level-wise и leaf-wise

Level-wise (по уровням)



- Раскрываются все узлы уровня сразу, дерево остается сбалансированным
- Глубина – естественный и понятный ограничитель сложности
- Часть разбиений делается «для симметрии» и почти не уменьшает loss

Leaf-wise (по лучшему листу)



- На каждом шаге раскрывается лист с максимальным приростом, где бы он ни находился
- При равном числе листьев loss ниже, чем у level-wise
- Дерево растёт вглубь неравномерно - на малых выборках легко переобучается

Чем ограничивают leaf-wise: num_leaves (главный параметр сложности вместо глубины), min_data_in_leaf, max_depth.

Часть 4. CatBoost: категориальные признаки и сдвиг предсказания

Target statistics и утечка таргета

Категорию заменяют на статистику таргета внутри неё, сглаженную к априорному значению p:

$$\hat{x}_i^k = \frac{\sum_{j=1}^{n} \mathbb{1}\{x_j^k = x_i^k\} \cdot y_j + a\,p}{\sum_{j=1}^{n} \mathbb{1}\{x_j^k = x_i^k\} + a}$$

Где ломается

$$\mathbb{E}\left[\hat{x}_i^k \mid y_i\right] \neq \mathbb{E}\left[\hat{x}^k \mid y\right]_{\text{test}}$$

- Объект участвует в вычислении собственного признака: y_i входит в числитель
- Предельный случай – категория, встреченная один раз: признак равен самому таргету
- Дерево находит «идеальное» разбиение на обучении, которое не воспроизводится на тесте

Методы починки и их цена

Holdout

Статистику считаем на отдельной части данных. Утечки нет, но и данных для обучения стало меньше.

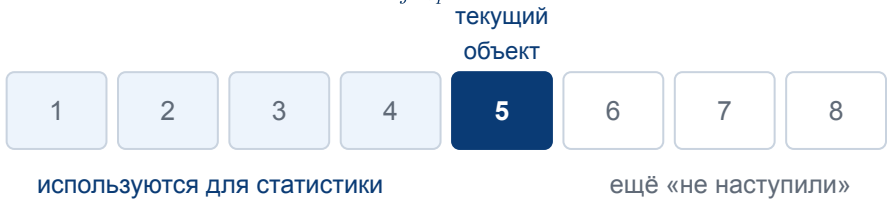
Leave-one-out

Исключаем сам объект. Кажется аккуратным, но информация о y_i просачивается через остальные значения – разбиение опять восстанавливает таргет.

Ordered Target Statistics: искусственное время

Вводим случайную перестановку объектов σ и считаем статистику только по тем, кто стоит раньше – как если бы данные приходили последовательно:

$$\hat{x}^k_{\sigma_p} = \frac{\sum_{j<p} \mathbb{1}\{x^k_{\sigma_j} = x^k_{\sigma_p}\} \cdot y_{\sigma_j} + a\,p}{\sum_{j<p} \mathbb{1}\{x^k_{\sigma_j} = x^k_{\sigma_p}\} + a}$$



Что это даёт

- Собственный таргет объекта в его признак не входит — утечки нет
- В отличие от holdout, используются все обучающие данные

Чем платим

- У ранних объектов статистика построена по нескольким наблюдениям – высокая дисперсия
- Поэтому используется несколько независимых перестановок, а не одна

Для реальных временных данных перестановка не нужна – там уже есть настоящее время, и порядок надо брать из него.

Prediction shift и ordered boosting

Та же проблема шире категориальных признаков. Градиент g_i считается по модели F_{i-1}, которая обучалась в том числе на объекте i:

$$F_{m-1}(x_i) \mid x_i \in \mathcal{D} \quad \not\sim \quad F_{m-1}(x) \mid x \sim P$$

распределение предсказаний на обучающем объекте отличается от распределения на новом

Почему это накапливается

- Смещённые градиенты дают смещённое дерево
- Следующая итерация считает градиенты уже по смещённой модели
- Итог – систематический разрыв между обучением и тестом, который не лечится ранней остановкой

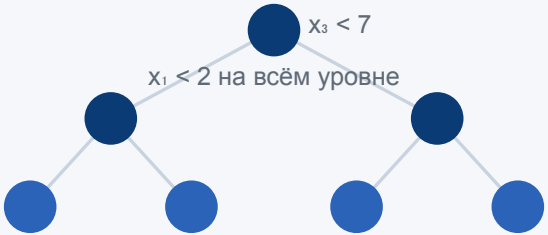
Ordered boosting

- Тот же порядок σ: градиент объекта считается по модели, обученной только на предшествующих объектах
- Наивная реализация потребовала бы n отдельных моделей – на практике используются логарифмически много «срезов» и общие перестановки
- Режим Ordered заметно помогает на небольших выборках; на больших данных Plain обычно достаточно

Ordered TS и ordered boosting – один и тот же принцип, примененный к признакам и к градиентам соответственно.

Симметричные деревья и комбинации категорий

Oblivious (симметричные) деревья



- На каждом уровне – одно и то же условие для всех узлов
- Дерево эквивалентно таблице из 2^depth ячеек: индекс листа собирается битовыми операциями
- Инференс становится очень быстрым, а сама модель - сильно регуляризованной

Комбинации категориальных признаков

- Взаимодействие категорий дерево «по одному признаку за раз» ловит плохо
- CatBoost жадно строит комбинации прямо во время роста дерева: к уже использованным в ветке категориям добавляется новая
- Комбинация трактуется как новая категория, к которой применяется тот же ordered TS
- Число комбинаций ограничивается, иначе их становится экспоненциально много

Где CatBoost силён: много категорий высокой мощности (кардинальности), небольшие и средние выборки, жёсткие требования к скорости инференса.

Часть 5. Feature engineering: target encoding и пайплайны

Target encoding: сглаживание к глобальному среднему

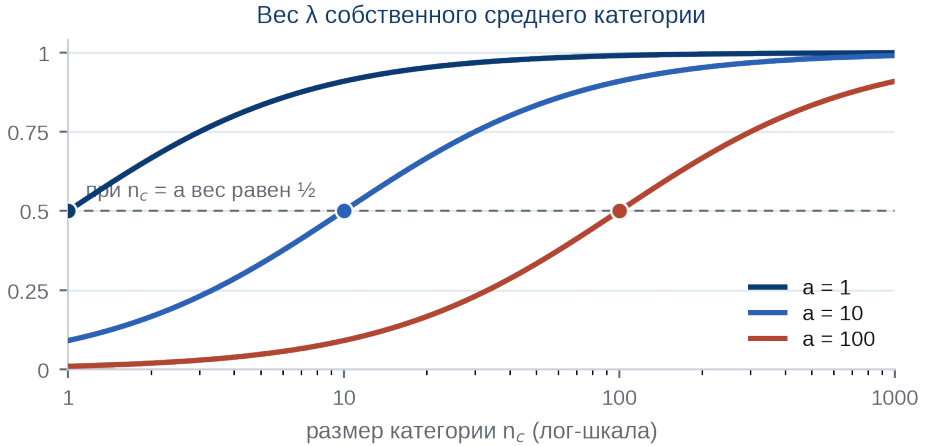
$$\mathrm{TE}(c) = \frac{n_c \cdot \bar{y}_c + a \cdot \bar{y}}{n_c + a}$$

n_c — число объектов категории, $\bar{y}_c$ — среднее таргета в ней, $\bar{y}$ — глобальное среднее, a — сила сглаживания

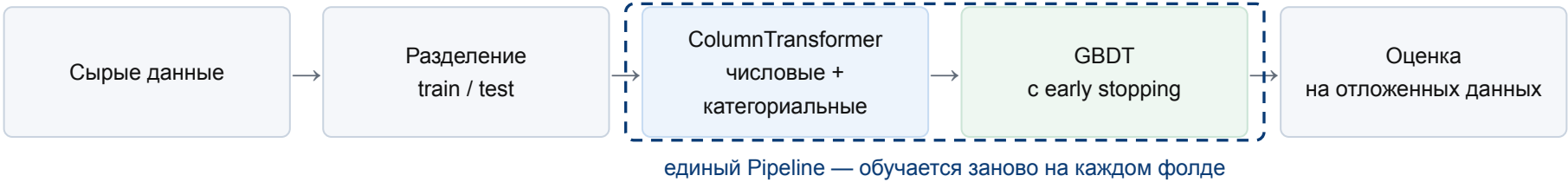
$$\lambda(n_c) = \frac{n_c}{n_c + a} \qquad \mathrm{TE}(c) = \lambda \bar{y}_c + (1 - \lambda)\bar{y}$$

Как читать

- Кодировка – выпуклая комбинация среднего по категории и глобального среднего
- Редкая категория почти полностью стягивается к ȳ, частая - к своему среднему
- a задаёт, при каком размере категории мы начинаем ей доверять: при n_c = a вес равен ½



Пайплайн без утечек



Ключевые правила

- Любая операция, использующая таргет, живёт внутри пайплайна и переобучается на каждом фолде
- Early stopping - по отдельной валидационной части, никогда по тесту
- Тест открывается один раз, в самом конце
- Для временных данных - только TimeSeriesSplit и расширяющееся окно

Обработка категорий у разных GB

CatBoost	категории отдаём библиотеке через cat_features - ordered TS уже внутри
LightGBM	есть встроенная обработка категорий, но контроль сглаживания слабее
XGBoost	кодирование делаем сами и обязательно внутри пайплайна

XGBoost, LightGBM, CatBoost: где они действительно различаются

	XGBoost	LightGBM	CatBoost
Поиск разбиений	exact greedy и гистограммы	гистограммы, GOSS, EFB	гистограммы
Рост дерева	по уровням (есть lossguide)	по лучшему листу	симметричные деревья
Категории	кодируем сами	встроенная обработка	ordered TS и комбинации
Основная регуляризация	λ, α, γ, сэмплирование	num_leaves, min_data_in_leaf	ordered boosting, симметрия
Сильная сторона	зрелость и детальный контроль	скорость на больших выборках	категории и качество из коробки
Слабое место	нужна ручная работа с категориями	легко переобучить на малых данных	обучение медленнее

Математическое ядро у всех трёх одно и то же - ньютоновский шаг из части 1. Различия начинаются на уровне того, как ищутся разбиения и как обрабатываются категории.

Источники

1. Chen, T., Guestrin, C. (2016). “XGBoost: A Scalable Tree Boosting System.” KDD.
2. Ke, G. et al. (2017). “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” NeurIPS 30.
3. Prokhorenkova, L. et al. (2018). “CatBoost: Unbiased Boosting with Categorical Features.” NeurIPS 31.
4. Dorogush, A. V., Ershov, V., Gulin, A. (2018). “CatBoost: Gradient Boosting with Categorical Features Support.” arXiv:1810.11363.
5. Friedman, J. H. (2001). “Greedy Function Approximation: A Gradient Boosting Machine.” The Annals of Statistics.
6. Nielsen, D. (2016). Tree Boosting With XGBoost: Why Does XGBoost Win Every Machine Learning Competition? MSc thesis, NTNU.
7. Sigrist, F. (2021). “Gradient and Newton Boosting for Classification and Regression.” Expert Systems with Applications.
8. Micci-Barreca, D. (2001). “A Preprocessing Scheme for High-Cardinality Categorical Attributes.” ACM SIGKDD Explorations.
9. Hastie, T., Tibshirani, R., Friedman, J. (2009). The Elements of Statistical Learning. Springer, Chapter 10.
10. Pedregosa, F. et al. (2011). “Scikit-learn: Machine Learning in Python.” JMLR — разделы Pipeline и TargetEncoder.